UNIVERSITÉ ALIOUNE DIOP DE BAMBEY
UFR Sciences & Technologies | Master 2 Big Data & Intelligence Artificielle
Année universitaire 2025-2026

PROJET TUTORÉ — CODES SOURCES COMPLETS

🚌 Dakar Smart City

Système Temps-Réel de Mobilité Urbaine

BRT • Dakar Dem Dikk • Taxis • Clandos • MLOps

Durée	4 semaines
Équipe	2 étudiants
Domaine	Big Data Streaming, NoSQL & Machine Learning — Mobilité Urbaine
Standard	M2 Approche Industrie | Dakar Smart City 2035
Enseignant	Mr Ahmed Ben Sidy Bouya SEYE — Senior Big Data & AI Engineer | Groupe Sonatel

NiFi • Kafka • Spark Streaming • Pandera • HBase • Hive • Airflow • RandomForest • Privacy by Design

1. Contexte et Objectifs

Dakar compte 4 millions d'habitants et un réseau de transport complexe : BRT (Bus Rapid Transit inauguré 2023), Dakar Dem Dikk (370 bus), taxis, clandos et cars rapides. Les embouteillages coûtent ~150 milliards FCFA/an en perte de productivité. Ce projet construit une plateforme de supervision temps réel pour le CETUD, permettant de détecter les congestions, prédire les vitesses de circulation et optimiser le déploiement des véhicules.

> ℹ️ **Dakar Smart City 2035**
> Ce projet s'inscrit dans le Plan Directeur d'Urbanisme de Dakar 2025-2035 et le projet Smart City Sénégal financé par la BID. L'intégration du BRT Dakar (ligne 1 : Petersen–Guédiawaye, 18,3 km, 14 stations) avec les données temps réel est un axe stratégique du New Deal Technologique.

1.1 Objectifs Techniques

#	Composant	Description
1	**Ingénierie Qualité**	Validation Pandera : coordonnées GPS bounding box Dakar + format plaque sénégalais (XX-9999-X)
2	**Ingestion Orchestrée**	NiFi Process Group 'Dakar_Mobility_Ingestion' : ListenHTTP/GetFile → UpdateAttribute → PublishKafka
3	**Privacy by Design**	SHA-256 de la plaque immatriculée + sel DAKAR_SECRET_SALT + drop(vehicle_id) avant stockage
4	**Spark Streaming**	2 queries indépendantes : congestion agrégée (fenêtre 15min) + Gold Layer Hive (append)
5	**MLOps Versionnage**	RandomForest vitesse prédite — versionnage timestamp + Model Registry HDFS + lien 'latest'
6	**Monitoring Drift**	RMSE horaire via vue Hive — BranchPythonOperator : RMSE > 15 km/h → réentraînement auto

1.2 Architecture End-to-End

```
 [ SOURCE ]  ──(Pandera)──▶ [ NiFi ]  ──▶ [ KAFKA ]  ──▶ [ PROCESSING ]  ──▶ [ STORAGE ]
  Simulateur                 Orchest.      Topic:         Spark Streaming     HBase
(NoSQL)
  Python/GPS  Validation      Data Flow      dakar_         + Inférence ML      temps réel
  Véhicules   Contrat                        mobility_      Privacy Layer       Gold Layer
                                             clean          SHA-256                  │
                                                                  │            [ ANALYTICS
]
       [ MLOps LOOP ] ◀──── (Drift RMSE>15) ◀──── [ MONITORING ] Apache Hive
        Airflow DAG          Réentraînement         Hive / HBase  vue_etat_trafic
        Horaire              spark-submit           Drift history  vue_model_drift
        BranchPython         train_and_catalog.py                  vue_perf_lignes
```

2. Prérequis & Installation

ℹ️ Configuration requise

RAM : 16 Go. CPU : 4 cœurs. Ubuntu 22.04 LTS. Docker Engine 24.x. Java 11 (Spark). Interface NiFi accessible sur http://localhost:8081 pour la soutenance (obligatoire).

setup.sh

```
# Setup complet Dakar Smart City
mkdir -p data dags models scripts nifi_templates
docker compose up -d zookeeper && sleep 10
docker compose up -d kafka nifi hbase hive-metastore && sleep 30
docker compose up -d spark-master spark-worker airflow

# Initialiser HBase
python hbase_setup.py

# Créer les tables Hive
docker exec hive-metastore hive -f hive_setup.sql

# Démarrer le simulateur GPS (en arrière-plan)
python kafka_producer_dakar.py &

# Lancer le pipeline Spark Streaming
spark-submit --master spark://spark-master:7077 streaming_core.py

# Entraînement initial du modèle (après accumulation de données)
spark-submit --master spark://spark-master:7077 scripts/train_and_catalog.py

# Interfaces :
#   NiFi    : http://localhost:8081  (Process Group : Dakar_Mobility_Ingestion)
#   Spark   : http://localhost:8080
#   HBase   : http://localhost:16010
#   Airflow : http://localhost:8082  (DAG : dakar_smart_city_monitoring)
```

requirements.txt

```
# requirements.txt — Dakar Smart City
pyspark==3.3.2
kafka-python==2.0.2
pandera==0.17.2
pandas==2.0.3
apache-airflow==2.7.3
mlflow==2.8.1
pyhive==0.7.0
happybase==1.2.0
matplotlib==3.7.3
seaborn==0.12.2
scipy==1.11.4
numpy==1.24.4
faker==30.1.0
```

3. Codes Sources Complets & Fonctionnels

3.1 Infrastructure Docker — docker-compose.yml

Le sel DAKAR_SECRET_SALT est injecté dans les containers Spark master et worker via les variables d'environnement Docker. Il n'apparaît jamais en clair dans les scripts Python.

docker-compose.yml

```yaml
version: '3.8'

services:
  zookeeper:
    image: bitnami/zookeeper:3.8
    container_name: zookeeper
    environment:
      ALLOW_ANONYMOUS_LOGIN: "yes"
    networks: [dakar_net]

  kafka:
    image: bitnami/kafka:3.3
    container_name: kafka
    depends_on: [zookeeper]
    ports: ["9092:9092"]
    environment:
      KAFKA_CFG_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_CFG_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
      ALLOW_ANONYMOUS_LOGIN: "yes"
      KAFKA_CFG_AUTO_CREATE_TOPICS_ENABLE: "true"
    networks: [dakar_net]

  nifi:
    image: apache/nifi:1.23.2
    container_name: nifi
    ports: ["8081:8080"]
    environment:
      NIFI_WEB_HTTP_PORT: 8080
    volumes:
      - ./nifi_templates:/opt/nifi/nifi-current/conf/templates
    networks: [dakar_net]

  spark-master:
    image: bitnami/spark:3.3
    container_name: spark-master
    environment:
      SPARK_MODE: master
      DAKAR_SECRET_SALT: UADB_M2_2025_SECURE_SALT
    ports: ["8080:8080", "7077:7077"]
    volumes:
      - ./models:/opt/uadb/models
      - ./scripts:/opt/uadb/scripts
    networks: [dakar_net]

  spark-worker:
    image: bitnami/spark:3.3
    container_name: spark-worker
    depends_on: [spark-master]
    environment:
      SPARK_MODE: worker
      SPARK_MASTER_URL: spark://spark-master:7077
      DAKAR_SECRET_SALT: UADB_M2_2025_SECURE_SALT
    networks: [dakar_net]

  hbase:
    image: harisekhon/hbase:2.1
    container_name: hbase
    ports: ["16010:16010", "9090:9090"]
    volumes:
      - hbase_data:/data
    networks: [dakar_net]
```

```
  hive-metastore:
    image: apache/hive:3.1.3
    container_name: hive-metastore
    environment:
      SERVICE_NAME: metastore
    networks: [dakar_net]

  airflow:
    image: apache/airflow:2.7.3
    container_name: airflow
    ports: ["8082:8080"]
    volumes:
      - ./dags:/opt/airflow/dags
      - ./models:/opt/uadb/models
    networks: [dakar_net]

volumes:
  hbase_data:
    driver: local

networks:
  dakar_net:
    driver: bridge
```

3.2 Initialisation HBase — hbase_setup.py

> ⚠ **Fichier obligatoire**
> 3 tables créées : dakar:vehicules_temps_reel (TTL 1h), dakar:alertes_trafic (TTL 30min), dakar:model_drift (30 jours de RMSE horaire).

hbase_setup.py

```
# hbase_setup.py — Tables HBase Dakar Smart City
import happybase, logging
logger = logging.getLogger('HBaseSetup')

def create_dakar_tables():
    conn = happybase.Connection('hbase', port=9090, timeout=10000)
    conn.open()
    tables = {
        # Positions GPS temps réel (TTL 1h — données opérationnelles)
        b'dakar:vehicules_temps_reel': {
            b'gps':     {'max_versions': 1, 'time_to_live': 3600},
            b'predict': {'max_versions': 5},   # vitesse_predite, conf_score
            b'meta':    {'max_versions': 1},   # vehicle_secure, type, ligne
        },
        # Alertes congestion actives (TTL 30min)
        b'dakar:alertes_trafic': {
            b'alerte':  {'max_versions': 10, 'time_to_live': 1800},
        },
        # Historique drift modèle (30 jours de RMSE horaire)
        b'dakar:model_drift': {
            b'metrics': {'max_versions': 720},  # 30j × 24h
        },
    }
    existantes = [t.decode() for t in conn.tables()]
    for nom_b, fam in tables.items():
        nom = nom_b.decode()
        if nom not in existantes:
            conn.create_table(nom_b, fam)
            logger.info(f'Table {nom} créée ✓')
        else:
            logger.info(f'Table {nom} déjà existante')
    conn.close()

if __name__ == '__main__':
    logging.basicConfig(level=logging.INFO)
    create_dakar_tables()
```

3.3 Contrat de Données Pandera — schema.py

> ✓ **Contraintes GPS Dakar**
> Bounding box : lat [14.60, 14.85] × lon [-17.55, -17.30]. Format plaque sénégalais validé par regex : ^[A-Z]{2}-\d{4}-[A-Z]$. Coerce=True pour les timestamps pandas.

schema.py

```
# schema.py — Contrat de données Dakar Smart City
# Validation Pandera : coordonnées GPS Dakar + plaques sénégalaises

import pandera as pa
import pandas as pd
from pandera.typing import Series

# Types de véhicules Dakar (CETUD / Dakar Dem Dikk)
VEHICLE_TYPES = ['Bus_DDD', 'Taxi', 'BRT', 'Clandos', 'Car_Rapide']
```

```
class DakarMobilitySchema(pa.SchemaModel):
    """Validation stricte des données GPS véhicules Dakar."""
    # Format plaque sénégalais : ex. DK-1234-A
    vehicle_id:   Series[str]          = pa.Field(
                      str_matches=r'^[A-Z]{2}-\d{4}-[A-Z]$')
    vehicle_type: Series[str]          = pa.Field(isin=VEHICLE_TYPES)
    # Bounding box Grand Dakar (Cap-Vert péninsule + banlieue)
    latitude:     Series[float]        = pa.Field(ge=14.60, le=14.85)
    longitude:    Series[float]        = pa.Field(ge=-17.55, le=-17.30)
    speed_kmh:    Series[float]        = pa.Field(ge=0.0, le=120.0)
    heading_deg:  Series[float]        = pa.Field(ge=0.0, le=360.0)
    ligne_id:     Series[str]          = pa.Field()  # ex. 'BRT-01', 'DDD-15'
    passagers:    Series[int]          = pa.Field(ge=0, le=100)
    timestamp:    Series[pd.Timestamp] = pa.Field(nullable=False)

    class Config:
        coerce = True
        strict = True

def validate_and_filter(df: pd.DataFrame) -> pd.DataFrame:
    """Valide et filtre les enregistrements GPS invalides."""
    try:
        return DakarMobilitySchema.validate(df, lazy=True)
    except pa.errors.SchemaErrors as exc:
        err = exc.failure_cases
        import logging
        logging.warning(f'[DakarMobility] {len(err)} GPS invalide(s) rejeté(s)')
        valid_idx = df.index.difference(err['index'].dropna().astype(int))
        return df.loc[valid_idx]
```

3.4 Simulateur GPS — kafka_producer_dakar.py

Simule 6 lignes de transport Dakar avec trajectoires réelles, zones de congestion géolocalisées (Plateau, Colobane, VDN, Guédiawaye), et modélisation des heures de pointe (7-9h, 17-20h). Vitesse réduite à ~8 km/h en heure de pointe vs ~25 km/h hors pointe.

kafka_producer_dakar.py

```
# kafka_producer_dakar.py — Simulateur GPS véhicules Dakar
# BRT, Dakar Dem Dikk, Taxis, Clandos — avec saisonnalité heure de pointe
from kafka import KafkaProducer
import json, random, time, uuid
from datetime import datetime
import numpy as np

random.seed(42); np.random.seed(42)

# Lignes de transport Dakar avec parcours réels (approximatif)
LIGNES = {
    'BRT-01':   {'type':'BRT',      'nb_vehicules':12, 'depart':(14.693,-17.447),
'arrivee':(14.772,-17.347)},
    'DDD-15':   {'type':'Bus_DDD',  'nb_vehicules':20, 'depart':(14.680,-17.440),
'arrivee':(14.730,-17.400)},
    'DDD-22':   {'type':'Bus_DDD',  'nb_vehicules':15, 'depart':(14.693,-17.447),
'arrivee':(14.760,-17.390)},
    'TAXI-PL':  {'type':'Taxi',     'nb_vehicules':50, 'depart':(14.693,-17.447),
'arrivee':(14.750,-17.380)},
    'CLAN-01':  {'type':'Clandos',  'nb_vehicules':30, 'depart':(14.710,-17.430),
'arrivee':(14.775,-17.360)},
    'CAR-GD':   {'type':'Car_Rapide','nb_vehicules':10,'depart':(14.680,-17.450),
'arrivee':(14.760,-17.370)},
}

# Positions des zones de congestion connues (Pont de l'Émergence, VDN, Colobane...)
ZONES_CONGESTION = [
```

```
    (14.693, -17.447),  # Plateau / Centre-ville
    (14.720, -17.430),  # Colobane
    (14.744, -17.410),  # VDN Pikine
    (14.760, -17.395),  # Guédiawaye
]

producer = KafkaProducer(
    bootstrap_servers=['kafka:9092'],
    value_serializer=lambda v: json.dumps(v).encode()
)

def near_congestion(lat, lon) -> bool:
    """Vérifie si le véhicule est dans un rayon de 500m d'une zone de congestion."""
    for clat, clon in ZONES_CONGESTION:
        dist = ((lat - clat)**2 + (lon - clon)**2) ** 0.5
        if dist < 0.005:  # ~500m en degrés
            return True
    return False

def gen_gps_event(ligne_id: str) -> dict:
    """Génère un événement GPS réaliste pour un véhicule d'une ligne."""
    ligne  = LIGNES[ligne_id]
    h      = datetime.utcnow().hour
    # Heures de pointe : 7-9h et 17-20h → vitesse réduite, plus de passagers
    heure_pointe = (7 <= h <= 9) or (17 <= h <= 20)
    speed_base   = 8  if heure_pointe else 25   # km/h (embouteillages)
    pass_base    = 55 if heure_pointe else 25

    # Position interpolée entre départ et arrivée
    t   = random.random()  # 0..1 = position sur le trajet
    lat = ligne['depart'][0] + t * (ligne['arrivee'][0] - ligne['depart'][0])
    lon = ligne['depart'][1] + t * (ligne['arrivee'][1] - ligne['depart'][1])
    # Bruit GPS réaliste (±50m)
    lat += np.random.normal(0, 0.0005)
    lon += np.random.normal(0, 0.0005)

    # Congestion locale → vitesse encore réduite
    if near_congestion(lat, lon):
        speed_base = max(2, speed_base * 0.4)

    return {
        'vehicle_id':   f'{random.choice(["DK","TH","KL"])}-{random.randint(1000,9999)}-
{random.choice("ABCDE")}',
        'vehicle_type': ligne['type'],
        'latitude':     round(max(14.60, min(14.85, lat)), 6),
        'longitude':    round(max(-17.55, min(-17.30, lon)), 6),
        'speed_kmh':    round(max(0, np.random.normal(speed_base, 3)), 1),
        'heading_deg':  round(random.uniform(0, 360), 1),
        'ligne_id':     ligne_id,
        'passagers':    min(100, max(0, int(np.random.normal(pass_base, 10)))),
        'timestamp':    datetime.utcnow().strftime('%Y-%m-%dT%H:%M:%SZ'),
    }

if __name__ == '__main__':
    print('Simulateur Dakar Smart City démarré...')
    while True:
        for ligne_id in LIGNES:
            producer.send('dakar_mobility_clean', gen_gps_event(ligne_id))
        producer.flush()
        time.sleep(2)
```

3.5 Pipeline Spark Streaming — streaming_core.py

> ⚠️ **Points critiques**
> (1) drop('vehicle_id') OBLIGATOIRE après SHA-256. (2) COALESCE sur toutes les colonnes nullables. (3) 2 queries indépendantes : congestion → Kafka + données enrichies → Hive. (4) Fallback modèle = règle experte si model/latest absent.

streaming_core.py

```
# streaming_core.py — Pipeline Spark Streaming Dakar Smart City
# Privacy Layer + Inférence vitesse + Détection congestion
import os
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, sha2, concat, lit, from_json, to_json, struct,
    window, avg, count, when, coalesce, current_timestamp
)
from pyspark.sql.types import (
    StructType, StructField, StringType, FloatType, IntegerType
)
from pyspark.ml import PipelineModel

SECRET_SALT = os.environ.get('DAKAR_SECRET_SALT', 'fallback_salt_uadb_2025')
BROKERS     = 'kafka:9092'
MODEL_PATH  = '/opt/uadb/models/latest'

spark = SparkSession.builder \
    .appName('DakarSmartCity_Streaming') \
    .config('spark.sql.shuffle.partitions', '4') \
    .enableHiveSupport() \
    .getOrCreate()
spark.sparkContext.setLogLevel('WARN')

# Schéma GPS véhicules
gps_schema = StructType([
    StructField('vehicle_id',   StringType(), True),
    StructField('vehicle_type', StringType(), True),
    StructField('latitude',     FloatType(),  True),
    StructField('longitude',    FloatType(),  True),
    StructField('speed_kmh',    FloatType(),  True),
    StructField('heading_deg',  FloatType(),  True),
    StructField('ligne_id',     StringType(), True),
    StructField('passagers',    IntegerType(),True),
    StructField('timestamp',    StringType(), True),
])

# ── Lecture Kafka ──────────────────────────────────────────────
raw_df = (spark.readStream.format('kafka')
    .option('kafka.bootstrap.servers', BROKERS)
    .option('subscribe', 'dakar_mobility_clean')
    .option('startingOffsets', 'latest')
    .load()
    .select(from_json(col('value').cast('string'), gps_schema).alias('d'))
    .select('d.*')
    .withColumn('event_ts', current_timestamp())
)

# ── Privacy Layer ──────────────────────────────────────────────
# Hachage SHA-256 de la plaque — irréversible sans le sel
secure_df = (
    raw_df
    .withColumn('vehicle_secure',
                sha2(concat(col('vehicle_id'), lit(SECRET_SALT)), 256))
    .drop('vehicle_id')  # Suppression OBLIGATOIRE de la plaque brute
    # Protection contre les NULL GPS
    .withColumn('latitude',  coalesce(col('latitude'),  lit(14.693)))
    .withColumn('longitude', coalesce(col('longitude'), lit(-17.447)))
    .withColumn('speed_kmh', coalesce(col('speed_kmh'), lit(0.0)))
    .withColumn('passagers', coalesce(col('passagers'), lit(0)))
)
```

```
# ── Inférence ML : prédiction vitesse attendue ──────────────────
try:
    model = PipelineModel.load(MODEL_PATH)
    scored_df = model.transform(secure_df)
    print('Modèle chargé depuis', MODEL_PATH)
except Exception as e:
    print(f'⚠️ Modèle non disponible ({e}) — mode règles expertes')
    # Fallback : règle experte basée sur l'heure
    from pyspark.sql.functions import hour
    scored_df = secure_df.withColumn('speed_predicted',
        when((hour(current_timestamp()).between(7,9)) |
             (hour(current_timestamp()).between(17,20)), lit(8.0))
        .otherwise(lit(25.0)))

# ── Détection congestion (fenêtre 15min, watermark 5min) ────────
congestion_df = (
    scored_df
    .withWatermark('event_ts', '5 minutes')
    .groupBy(window('event_ts', '15 minutes', '5 minutes'), 'ligne_id')
    .agg(
        avg('speed_kmh').alias('vitesse_moy'),
        avg(coalesce('speed_predicted', lit(25.0))).alias('vitesse_predite'),
        count('vehicle_secure').alias('nb_vehicules'),
        avg('passagers').alias('passagers_moy'),
    )
    .withColumn('ecart_vitesse',
        coalesce(col('vitesse_predite'), lit(25.0)) -
        coalesce(col('vitesse_moy'),     lit(25.0)))
    .withColumn('niveau_congestion',
        when(col('vitesse_moy') < 5,  lit('CRITIQUE'))
        .when(col('vitesse_moy') < 15, lit('MODERE'))
        .otherwise(lit('FLUIDE')))
)

# ── Query 1 : Alertes trafic → Kafka ────────────────────────────
q1 = (
    congestion_df
    .select(to_json(struct('*')).alias('value'))
    .writeStream.format('kafka')
    .option('kafka.bootstrap.servers', BROKERS)
    .option('topic', 'dakar_trafic_alerts')
    .option('checkpointLocation', '/tmp/dakar_congestion_ckpt')
    .outputMode('update')
    .start()
)

# ── Query 2 : Données enrichies → Hive (Gold Layer) ─────────────
q2 = (
    scored_df
    .select('vehicle_secure', 'vehicle_type', 'ligne_id',
            'latitude', 'longitude', 'speed_kmh',
            'passagers', 'event_ts')
    .writeStream.format('parquet')
    .option('path', 'hdfs:///dakar_smart_city/gold/')
    .option('checkpointLocation', '/tmp/dakar_gold_ckpt')
    .outputMode('append')
    .start()
)

q1.awaitTermination()
```

3.6 Entraînement & Versionnage — train_and_catalog.py

Features : latitude, longitude, passagers, heure, jour_semaine, vehicle_type_idx, ligne_id_idx. Modèle : RandomForest 100 arbres, maxDepth=8. Versionnage timestamp + lien 'latest' HDFS pour rollback. Métriques RMSE/R² sauvegardées en JSON.

scripts/train_and_catalog.py

```
# train_and_catalog.py — Entraînement + Versionnage modèle Dakar
# Prédit la vitesse attendue (speed_kmh) selon heure, ligne, type
from pyspark.sql import SparkSession
from pyspark.ml import Pipeline
from pyspark.ml.feature import StringIndexer, VectorAssembler, StandardScaler
from pyspark.ml.regression import RandomForestRegressor
from pyspark.ml.evaluation  import RegressionEvaluator
import datetime, subprocess

spark = SparkSession.builder \
    .appName('DakarSmartCity_Train') \
    .enableHiveSupport() \
    .getOrCreate()
spark.sparkContext.setLogLevel('WARN')

# ── Chargement données historiques (table Hive Gold) ──────────
df = spark.table('dakar_smart_city.mobilite_gold').cache()
print(f'Données chargées : {df.count()} enregistrements')

# ── Features ──────────────────────────────────────────────────
from pyspark.sql.functions import hour, dayofweek, col
df = (
    df
    .withColumn('heure',        hour(col('event_ts')))
    .withColumn('jour_semaine', dayofweek(col('event_ts')))
    .filter(col('speed_kmh').isNotNull())
)

NUM_COLS = ['latitude', 'longitude', 'passagers', 'heure', 'jour_semaine']
CAT_COLS = ['vehicle_type', 'ligne_id']

indexers  = [StringIndexer(inputCol=c, outputCol=c+'_idx', handleInvalid='skip')
             for c in CAT_COLS]
assembler = VectorAssembler(
    inputCols=NUM_COLS + [c+'_idx' for c in CAT_COLS],
    outputCol='unscaled_features', handleInvalid='skip'
)
scaler = StandardScaler(
    inputCol='unscaled_features', outputCol='features',
    withStd=True, withMean=True
)
rf = RandomForestRegressor(
    featuresCol='features', labelCol='speed_kmh',
    numTrees=100, maxDepth=8, seed=42
)

pipeline = Pipeline(stages=indexers + [assembler, scaler, rf])

# ── Entraînement ──────────────────────────────────────────────
train, test = df.randomSplit([0.8, 0.2], seed=42)
train.cache(); test.cache()
model = pipeline.fit(train)

# ── Évaluation ────────────────────────────────────────────────
preds    = model.transform(test)
ev_rmse  = RegressionEvaluator(labelCol='speed_kmh',
               predictionCol='prediction', metricName='rmse')
ev_r2    = RegressionEvaluator(labelCol='speed_kmh',
               predictionCol='prediction', metricName='r2')
rmse = ev_rmse.evaluate(preds)
r2   = ev_r2.evaluate(preds)
print(f'RandomForest | RMSE : {rmse:.2f} km/h | R² : {r2:.4f}')
```

```
# Feature importance
feat_names = NUM_COLS + CAT_COLS
importances = model.stages[-1].featureImportances.toArray()
for nm, imp in sorted(zip(feat_names[:len(importances)], importances),
                      key=lambda x:-x[1]):
    print(f'  {nm:30s} : {imp:.4f}')

# ── Versionnage du modèle (Model Registry HDFS) ─────────────────
version     = datetime.datetime.now().strftime('%Y%m%d_%H%M')
model_path  = f'/opt/uadb/models/traffic_v_{version}'
latest_path = '/opt/uadb/models/latest'

model.write().overwrite().save(model_path)
# Mettre à jour le lien 'latest' (symlink HDFS)
subprocess.run(['hdfs', 'dfs', '-rm', '-r', '-f', latest_path], check=False)
subprocess.run(['hdfs', 'dfs', '-cp', model_path, latest_path], check=True)
print(f'Modèle sauvegardé : {model_path}  (latest mis à jour)')

# Sauvegarder les métriques pour monitoring drift
import json
metrics = {'version': version, 'rmse': rmse, 'r2': r2}
with open(f'/opt/uadb/models/metrics_{version}.json', 'w') as f:
    json.dump(metrics, f)

train.unpersist(); test.unpersist(); df.unpersist()
spark.stop()
```

3.7 Tables Hive & Monitoring Drift — hive_setup.sql

La vue vue_model_drift implémente la requête SQL de monitoring demandée dans le PDF original. La vue vue_etat_trafic calcule l'état du trafic en temps réel sur les 15 dernières minutes.

hive_setup.sql

```
-- hive_setup.sql — Tables Hive Dakar Smart City
CREATE DATABASE IF NOT EXISTS dakar_smart_city
    COMMENT 'Mobilité Urbaine Dakar — UADB 2025-2026';
USE dakar_smart_city;

-- ── Table principale : Gold Layer mobilité ──────────────────────────
CREATE TABLE IF NOT EXISTS mobilite_gold (
    vehicle_secure  STRING  COMMENT 'SHA-256 — plaque anonymisée',
    vehicle_type    STRING,
    ligne_id        STRING,
    latitude        DOUBLE,
    longitude       DOUBLE,
    speed_kmh       DOUBLE,
    passagers       INT,
    event_ts        TIMESTAMP
)
PARTITIONED BY (date_obs STRING, ligne_part STRING)
STORED AS ORC TBLPROPERTIES ('orc.compress'='SNAPPY');

-- ── Table analytics : congestion par fenêtre ────────────────────────
CREATE TABLE IF NOT EXISTS congestion_historique (
    ligne_id          STRING,
    vitesse_moy       DOUBLE,
    vitesse_predite   DOUBLE,
    ecart_vitesse     DOUBLE,
    nb_vehicules      BIGINT,
    passagers_moy     DOUBLE,
    niveau_congestion STRING,
    periode_debut     TIMESTAMP
) STORED AS ORC;

-- ── Table monitoring drift ──────────────────────────────────────────
```

```
CREATE TABLE IF NOT EXISTS model_drift_history (
    periode       TIMESTAMP,
    rmse_horaire  DOUBLE,
    r2_horaire    DOUBLE,
    seuil_alerte  DOUBLE  DEFAULT 15.0
) STORED AS ORC;

-- ── Vue : Congestion en cours par ligne ──────────────────────────────
CREATE OR REPLACE VIEW vue_etat_trafic AS
SELECT
    ligne_id,
    AVG(COALESCE(speed_kmh, 0))           AS vitesse_moy_km_h,
    COUNT(vehicle_secure)                  AS nb_vehicules_actifs,
    AVG(COALESCE(passagers, 0))            AS passagers_moy,
    CASE
        WHEN AVG(speed_kmh) < 5  THEN 'CRITIQUE'
        WHEN AVG(speed_kmh) < 15 THEN 'MODERE'
        ELSE 'FLUIDE'
    END AS etat_trafic
FROM mobilite_gold
WHERE event_ts >= DATE_SUB(CURRENT_TIMESTAMP, 1/96.0)  -- 15 dernières minutes
GROUP BY ligne_id
ORDER BY vitesse_moy_km_h ASC;

-- ── Vue : Drift modèle (RMSE > seuil = réentraînement nécessaire) ────
CREATE OR REPLACE VIEW vue_model_drift AS
SELECT
    window(event_ts, '1 hour')             AS periode,
    AVG(ABS(COALESCE(speed_kmh,0) -
            COALESCE(speed_predicted,25))) AS current_rmse
FROM mobilite_gold
WHERE event_ts >= DATE_SUB(CURRENT_DATE(), 7)
GROUP BY window(event_ts, '1 hour')
HAVING current_rmse > 15
ORDER BY periode DESC;

-- ── Vue : Performance par ligne et type de véhicule ──────────────────
CREATE OR REPLACE VIEW vue_perf_lignes AS
SELECT
    ligne_id, vehicle_type,
    AVG(COALESCE(speed_kmh,0))  AS vitesse_moy,
    AVG(COALESCE(passagers,0))  AS taux_remplissage_moy,
    COUNT(vehicle_secure)       AS nb_enregistrements
FROM mobilite_gold
WHERE date_obs >= DATE_SUB(CURRENT_DATE(), 30)
GROUP BY ligne_id, vehicle_type
ORDER BY vitesse_moy ASC;
```

3.8 DAG Airflow MLOps — dakar_smart_city_dag.py

DAG horaire. T1 check_model_drift : requête Hive vue_model_drift, si > 3 périodes avec RMSE > 15 km/h → trigger_retrain. T2a trigger_retrain : spark-submit train_and_catalog.py. T2b update_hbase : alertes congestion CRITIQUE/MODERE → HBase. T3 update_hive_drift : persistance historique drift.

dags/dakar_smart_city_dag.py

```
# dags/dakar_smart_city_dag.py — DAG Airflow Dakar Smart City
# Monitoring drift RMSE + réentraînement automatique
from airflow import DAG
from airflow.operators.python import PythonOperator, BranchPythonOperator
from airflow.operators.dummy  import DummyOperator
from airflow.utils.dates      import days_ago
from datetime import timedelta
import subprocess, logging

logger = logging.getLogger('dakar_dag')
default_args = {
    'owner': 'seye_ahmed',
    'retries': 2,
    'retry_delay': timedelta(minutes=5),
    'email_on_failure': True,
    'email': ['dakar-smart-city@sonatel.sn'],
}

def check_model_drift(**ctx):
    """T1 — Vérifie le RMSE horaire via la vue Hive vue_model_drift.
    Si RMSE > 15 km/h sur plus de 3 périodes : déclencher réentraînement.
    """
    from pyhive import hive
    conn   = hive.Connection(host='hive-metastore', port=10000)
    cursor = conn.cursor()
    cursor.execute("""
        SELECT COUNT(*) AS nb_periodes_drift
        FROM dakar_smart_city.vue_model_drift
        WHERE current_rmse > 15
    """)
    nb_drift = cursor.fetchone()[0] or 0
    logger.info(f'Périodes en drift RMSE>15 : {nb_drift}')
    ctx['ti'].xcom_push(key='nb_drift', value=nb_drift)
    return 'trigger_retrain' if nb_drift > 3 else 'update_hbase'

def trigger_retrain(**ctx):
    """T2a — Réentraîne le modèle et met à jour le lien 'latest'.
    Lance spark-submit sur train_and_catalog.py.
    """
    result = subprocess.run(
        ['spark-submit', '--master', 'spark://spark-master:7077',
         '/opt/uadb/scripts/train_and_catalog.py'],
        capture_output=True, text=True, timeout=3600
    )
    if result.returncode != 0:
        raise Exception(f'Spark train failed: {result.stderr[-500:]}')
    logger.info('Modèle réentraîné et versionné ✓')

def update_hbase(**ctx):
    """T2b — Met à jour HBase avec les alertes de congestion actuelles.
    Lit vue_etat_trafic et pousse dans dakar:alertes_trafic.
    """
    from pyhive import hive
    import happybase
    from datetime import datetime

    conn_hbase  = happybase.Connection('hbase', port=9090)
    conn_hbase.open()
    conn_hive   = hive.Connection(host='hive-metastore', port=10000)
    cursor      = conn_hive.cursor()
    cursor.execute("""
        SELECT ligne_id, vitesse_moy_km_h, nb_vehicules_actifs, etat_trafic
        FROM dakar_smart_city.vue_etat_trafic
```

```
    """)
    table = conn_hbase.table(b'dakar:alertes_trafic')
    ts    = datetime.utcnow().isoformat()
    for row in cursor.fetchall():
        ligne, vitesse, nb_v, etat = row
        if etat in ('CRITIQUE', 'MODERE'):
            table.put(ligne.encode(), {
                b'alerte:ligne_id':   ligne.encode(),
                b'alerte:vitesse':    str(round(vitesse, 1)).encode(),
                b'alerte:nb_vehic':   str(nb_v).encode(),
                b'alerte:etat':       etat.encode(),
                b'alerte:ts':         ts.encode(),
            })
    conn_hbase.close()
    logger.info('HBase mis à jour ✓')

def update_hive_drift(**ctx):
    """T3 — Calcule et persiste le RMSE horaire dans model_drift_history.
    Exécute un INSERT INTO depuis la vue vue_model_drift.
    """
    from pyhive import hive
    conn   = hive.Connection(host='hive-metastore', port=10000)
    cursor = conn.cursor()
    cursor.execute("""
        INSERT INTO dakar_smart_city.model_drift_history
        SELECT periode, current_rmse, 0.0 AS r2_horaire, 15.0
        FROM dakar_smart_city.vue_model_drift
    """)
    conn.close()

with DAG(
    'dakar_smart_city_monitoring',
    default_args=default_args,
    description='Monitoring trafic + drift RMSE + réentraînement RF',
    schedule_interval='0 * * * *',  # Chaque heure
    start_date=days_ago(1),
    catchup=False,
    tags=['dakar-smart-city', 'mobilite', 'mlops']
) as dag:

    start   = DummyOperator(task_id='start')
    end     = DummyOperator(task_id='end')

    t_drift = BranchPythonOperator(
        task_id='check_model_drift',
        python_callable=check_model_drift,
        provide_context=True
    )
    t_train = PythonOperator(
        task_id='trigger_retrain',
        python_callable=trigger_retrain,
        provide_context=True
    )
    t_hbase = PythonOperator(
        task_id='update_hbase',
        python_callable=update_hbase,
        provide_context=True
    )
    t_hive  = PythonOperator(
        task_id='update_hive_drift',
        python_callable=update_hive_drift,
        provide_context=True
    )

    start >> t_drift >> [t_train, t_hbase]
    t_train >> t_hbase >> t_hive >> end
    t_hbase >> t_hive >> end
```

3.9 Dashboard — dashboard_mobilite.py

> ✓ **Livrable 4 panneaux**
> (1) État trafic par ligne avec codes couleur ROUGE/ORANGE/VERT. (2) Taux de remplissage par type de véhicule. (3) Performance BRT vs Bus Dakar Dem Dikk. (4) Courbe drift RMSE avec zone rouge de dépassement de seuil.

dashboard_mobilite.py

```
# dashboard_mobilite.py — Tableau de bord Dakar Smart City
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
from pyhive import hive

conn = hive.Connection(host='hive-metastore', port=10000,
                       database='dakar_smart_city')

# ── Charger les données ──────────────────────────────────────
etat_df   = pd.read_sql('SELECT * FROM vue_etat_trafic', conn)
perf_df   = pd.read_sql('SELECT * FROM vue_perf_lignes', conn)
drift_df  = pd.read_sql('SELECT * FROM vue_model_drift LIMIT 48', conn)

# ── Dashboard 4 panneaux ─────────────────────────────────────
fig, axes = plt.subplots(2, 2, figsize=(16, 12))
fig.suptitle('Dakar Smart City — Dashboard Mobilité Urbaine',
             fontsize=16, fontweight='bold', color='#0D3B6E')

# Panneau 1 : État trafic par ligne (barplot vitesse)
colors_trafic = ['#DC2626' if e=='CRITIQUE' else '#F59E0B' if e=='MODERE'
                 else '#16A34A' for e in etat_df['etat_trafic']]
axes[0,0].barh(etat_df['ligne_id'], etat_df['vitesse_moy_km_h'],
               color=colors_trafic)
axes[0,0].axvline(x=15, color='orange', linestyle='--', label='Seuil modéré')
axes[0,0].axvline(x=5,  color='red',    linestyle='--', label='Seuil critique')
axes[0,0].set_xlabel('Vitesse moyenne (km/h)')
axes[0,0].set_title('État Trafic par Ligne (temps réel)')
axes[0,0].legend(fontsize=8)

# Panneau 2 : Taux de remplissage par type de véhicule
remplissage = perf_df.groupby('vehicle_type')['taux_remplissage_moy'].mean()
axes[0,1].bar(remplissage.index, remplissage.values,
              color=['#1565C0','#F57C00','#2E7D32','#C62828','#6A1B9A'])
axes[0,1].set_xlabel('Type de véhicule')
axes[0,1].set_ylabel('Passagers moyen')
axes[0,1].set_title('Taux de Remplissage Moyen par Type')
axes[0,1].tick_params(axis='x', rotation=30)

# Panneau 3 : Evolution vitesse BRT vs Bus_DDD (30 derniers jours)
for vtype in ['BRT', 'Bus_DDD']:
    sub = perf_df[perf_df['vehicle_type']==vtype].sort_values('ligne_id')
    if len(sub):
        axes[1,0].plot(sub['ligne_id'], sub['vitesse_moy'],
                       marker='o', label=vtype)
axes[1,0].set_xlabel('Ligne'); axes[1,0].set_ylabel('Vitesse moy (km/h)')
axes[1,0].set_title('Performance BRT vs Bus Dakar Dem Dikk')
axes[1,0].legend(); axes[1,0].tick_params(axis='x', rotation=30)

# Panneau 4 : Drift RMSE modèle (alertes dépassement seuil)
if len(drift_df):
    axes[1,1].plot(range(len(drift_df)), drift_df['current_rmse'],
                   marker='o', color='#C62828', linewidth=1.5)
    axes[1,1].axhline(y=15, color='orange', linestyle='--',
                      label='Seuil réentraînement (15 km/h)')
    axes[1,1].fill_between(range(len(drift_df)), drift_df['current_rmse'],
                           15, where=drift_df['current_rmse']>15,
                           color='red', alpha=0.3, label='Zone drift')
axes[1,1].set_xlabel('Période (heures)')
axes[1,1].set_ylabel('RMSE (km/h)')
```

```
axes[1,1].set_title('Monitoring Drift Modèle RF')
axes[1,1].legend(fontsize=8)

plt.tight_layout()
plt.savefig('dashboard_dakar_smart_city.png', dpi=150, bbox_inches='tight')
print('Dashboard sauvegardé : dashboard_dakar_smart_city.png ✓')
```

4. Configuration NiFi — Dakar_Mobility_Ingestion

> ℹ️ **Interface obligatoire en soutenance**
> L'interface NiFi doit être accessible sur http://localhost:8081. Le Process Group 'Dakar_Mobility_Ingestion' doit être visible et actif avec les 3 processeurs configurés.

Processeur	Paramètre clé	Valeur
GetFile / ListenHTTP	Input Directory / Port	/opt/nifi/data/ | Port 8888
UpdateAttribute	data.origin	dakar_sensor
UpdateAttribute	data.timestamp	${now():format('yyyy-MM-dd\'T\'HH:mm:ssZ')}
PublishKafka_2_6	Kafka Brokers	kafka:9092
PublishKafka_2_6	Topic Name	dakar_mobility_clean
PublishKafka_2_6	Message Key	${vehicle_id}

📄 **validation_nifi.sh**

```
# Validation NiFi post-configuration

# 1. Vérifier que le topic existe
docker exec kafka kafka-topics.sh --list --bootstrap-server kafka:9092
# Attendu : dakar_mobility_clean

# 2. Consommer 5 messages de test
docker exec kafka kafka-console-consumer.sh \
  --bootstrap-server kafka:9092 \
  --topic dakar_mobility_clean \
  --max-messages 5 \
  --from-beginning

# 3. Exporter le template NiFi (livrable obligatoire)
# Menu NiFi → Process Group 'Dakar_Mobility_Ingestion'
# Clic droit → Download flow definition
# → Sauvegarder : template_nifi_dakar_equipeX.xml
```

5. Livrables Attendus

#	Livrable	Contenu attendu
1	**Pipeline de Données**	Code source complet sur GitHub + template NiFi XML (Dakar_Mobility_Ingestion) exporté.
2	**Pipeline ML**	train_and_catalog.py + DAG Airflow dakar_smart_city_monitoring fonctionnel. RMSE/R^2 documentés.
3	**Rapport MLOps**	Analyse du drift : graphique RMSE 7 jours + identification du meilleur modèle versionné. Comparaison RandomForest vs règle experte.
4	**Dashboard Mobilité**	dashboard_dakar_smart_city.png (4 panneaux) + recommandations CETUD concrètes (ex: renforcer BRT-01 aux heures de pointe).

6. Barème de Notation — /20 points

> ℹ️ **Barème différencié 4 niveaux**
> Interface NiFi obligatoirement visible sur port 8081 en soutenance. Le DAG Airflow doit avoir au moins 1 run réussi. Bonus documentation +2 pts.

Critère	Pts	Insuffisant (0)	Passable (50%)	Bien (75%)	Excellent (100%)
Infrastructure & Setup NiFi	3 pts	Docker non démarré, NiFi absent	Docker up mais NiFi sans topic	NiFi configuré, topic dakar_mobility_clean créé	Process Group 'Dakar_Mobility_Ingestion' actif, messages JSON visibles sur http://localhost:8081
Validation & Privacy (Pandera/CDP)	4 pts	Aucune validation Pandera, vehicle_id en clair	Schéma partiel (pas de regex plaque)	Schema GPS complet + SHA-256 sans drop(vehicle_id)	DakarMobilitySchema complet + drop(vehicle_id) + 0 plaque brute dans HBase/Hive + COALESCE
Streaming & Kafka	3 pts	Pas de Spark Streaming	Lecture Kafka seule sans traitement	1 query fonctionnelle (congestion OU Gold Layer)	2 queries indépendantes + watermarks + fallback modèle opérationnel
MLOps & IA (Airflow/Spark ML)	6 pts	Aucun modèle, pas de DAG	Règle experte seule, pas de versionnage	RF entraîné, DAG sans branchement drift	RF + versionnage timestamp + BranchPython RMSE>15 + réentraînement auto + RMSE/R² documentés
Mirroring HBase/Hive & Analytics	4 pts	Ni HBase ni Hive configurés	Tables créées mais vides	Vues Hive opérationnelles, HBase sans données	vue_etat_trafic + vue_model_drift + alertes HBase CRITIQUE/MODERE en live + dashboard 4 panneaux
TOTAL	20 pts				

Bonus Documentation (jusqu'à +2 pts)

Élément	Points bonus
README.md complet avec captures NiFi + docker compose up	+0.5 pt
Commentaires de code en français + docstrings sur chaque fonction	+0.5 pt
MLflow : tracking RMSE/R² sur au moins 3 runs versionnés	+0.5 pt
Démo live alerte HBase CRITIQUE sur une ligne sans erreur en soutenance	+0.5 pt

> ⚠️ **Note Importante**

L'interface NiFi doit être accessible sur le port 8081 pour la soutenance. Le binôme doit démontrer le cycle complet : simulateur GPS → NiFi → Kafka → Spark → HBase → alerte congestion visible en live.

🚌 Bon courage pour Dakar Smart City !

"Always be a solution, never a problem."

Mr Ahmed Ben Sidy Bouya SEYE | Senior Big Data & AI Engineer | Groupe Sonatel